Rocksmith 2014 Remastered (Steam, appid 221680) on Linux/Proton: Real Tone Cable / microphone input never detected despite verified-correct audio delivery

Summary

On Linux (Steam Play / Proton), Rocksmith 2014 Remastered's audio input level meter (used on the mandatory first-launch calibration screen, the Tuner, and the Path/Input device picker) **never reacts to any audio signal**, regardless of which capture device is behind it. This has been verified across three independent, confirmed-working audio delivery paths using two different physical Real Tone Cables plus synthetic/ambient signal sources. A full `WINEDEBUG=+wasapi,+mmdevapi,+alsa` trace shows Wine correctly opening the capture stream, correctly cycling `IAudioCaptureClient::GetBuffer`/`ReleaseBuffer`, and correctly delivering real, continuous, non-silent audio data from the underlying ALSA device. No Wine-side error, silence flag, or failed call appears anywhere in the trace. This strongly suggests the defect is not in Wine's WASAPI/ALSA implementation as such, but in a **behavioral difference** (most likely in capture-stream timing/padding/position semantics) between Wine's WASAPI implementation and real Windows, which the game's own (closed-source) signal-analysis code does not tolerate.

This document exists to (a) serve as a submittable Wine/Proton bug report and (b) as the canonical technical record of the investigation, for whoever picks this up next.

Environment

- **OS**: Bazzite (Fedora Atomic/ostree), image `ghcr.io/ublue-os/bazzite-nvidia-open:stable`, version `44.20260713` (2026-07-13)

- **Desktop**: KDE Plasma, Wayland (`kwin_wayland`)
- **Audio server**: PipeWire (`pipewire-alsa`, `pipewire-pulseaudio` 1.6.8), no separate PulseAudio daemon
- **GPU**: NVIDIA GeForce RTX 3090, proprietary driver 610.43.03 (`nvidia-open` kernel module flavor)
- **Steam**: native Linux Steam client
- **Game**: Rocksmith 2014 Edition Remastered, Steam appid `221680`, legitimately purchased/installed (not a cracked/P2P build)
- **Compat tool**: official **Proton 10.0** (`proton_10` in Steam's `CompatToolMapping`) - see "Related issue" below for why this specific build is required
- **Game engine**: Gamebryo/NetImmerse-derived renderer (`NiDX9Renderer`, Direct3D9), Wwise audio engine using **PortAudio** for input device enumeration (`NSoundInputMgr_Win32::DetectInputDevices_PortAudio`, visible in the game's own `DumpAudioLog=1` output to `audiodump.txt`)

Hardware tested

Two physically different Real Tone Cables, to rule out a defective-unit explanation:

Cable	USB ID	Chipset	ALSA capture channels
Hercules-branded	`12ba:00ff` ("Licensed by Sony Computer Entertainment America Rocksmith Guitar Adapter")	-	**1 (mono)**
Generic/ OEM	`0d8c:0134` ("C-Media Electronics Inc. USB PnP Audio Device")	C-Media	**1 (mono)**

Both cables: - Enumerate cleanly on the USB bus with no errors (`lsusb`, `dmesg`), negotiate their native USB Full-Speed (12 Mbps) rate correctly whether plugged directly into the motherboard's root hub or through a third-party powered hub (tested: a Corsair keyboard's integrated USB hub) - ruling out a USB power/port-type problem. - Are correctly recognized and fully functional at the OS/host level: `arecord -l` lists them as normal ALSA capture cards; a direct `arecord -D hw:CARD=...` (or via PipeWire, `pactl`/`wpctl`) test captures real, correct audio. - Confirmed previously fully functional under Windows (same PC, prior to migrating to this Linux install) on every USB port tested.

This rules out defective hardware, USB power delivery, and port type as causes.

Related issue (already resolved, documented for completeness)

Before this bug could even be investigated, the game failed to launch at all: `Rocksmith2014.exe` (running under the wine-preloader) was killed by `SIGSYS/SYS_SECCOMP` on syscall `madvise`, caused by the seccomp sandbox of **Steam Linux Runtime 4.0** (appid `4183110`). Reproducible on Proton Experimental, Proton 11.0, and GE-Proton11-1 - all of which require Runtime 4.0. Builds using the older **Steam Linux Runtime 3.0 "Sniper"** (appid `1628350`) do not hit this: official **Proton 10.0** does not crash. Note: **GE-Proton10-34**, despite using the same Sniper runtime, was observed to crash with the same SIGSYS intermittently on separate occasions - only the official Proton 10.0 build has been consistently stable across repeated testing. This is a separate bug from the audio issue below and is not the focus of this report, but explains why Proton 10.0 specifically (not GE-Proton10-34, not anything requiring Runtime 4.0) is the compat tool used for all testing here.

Reproduction steps

1. Install Rocksmith 2014 Remastered (appid 221680) via Steam, pin `CompatToolMapping` to official Proton 10.0 (see above - required just to get the game running at all).
2. Connect a Real Tone Cable (or any mono USB Audio Class capture device - see findings below, the device type does not matter to the underlying bug).
3. Launch the game. On first launch it forces a "test your guitar" calibration screen showing a live input-level meter.
4. Observe: the meter never moves, and after a timeout the game displays "It sounds like your cable is unplugged. Connect a Rocksmith Real Tone Cable." - regardless of actual signal present at the input.
5. This same non-reaction is reproducible in **every** other in-game audio-level UI: the Tuner (Tools menu) and the Path/Input device picker (Tools > Options > hold Ctrl for "Path/Input", after entering the calibration screen's "Disconnected" bypass mode to reach the main menu) - see below for how the mandatory first-run gate was bypassed for further testing.

Bypassing the mandatory calibration gate for further testing

The first-launch calibration screen has an alternate "Disconnected" input mode (distinct from "Real Tone Cable" and "Microphone") that allows proceeding to the main menu without a passing calibration. This was used to reach Tools > Tuner and Tools > Options > Path/Input for further testing, all of which exhibit the identical "cable is unplugged" failure independent of the first-run gate.

Diagnostic methodology and evidence

All of the below was captured via `Rocksmith.ini`'s `DumpAudioLog=1` (writes `audiodump.txt` in the game's install directory with the game's own `NSoundInputMgr_Win32`/PortAudio trace) plus, for the final and most conclusive test, `WINEDEBUG=+wasapi,+mmdevapi,+alsa` combined with `PROTON_LOG=1` set via Steam launch options, which captures every Wine-side WASAPI/ALSA API call made by the game process.

1. Initial state: PortAudio enumerates zero input devices

With the default `pulse` wine audio driver, `audiodump.txt` showed `NSoundInputMgr_Win32::DetectInputDevices_PortAudio - no new input devices, stop` with **no devices ever found**, despite the host (PipeWire/ALSA) correctly capturing from the cable (`alsa_input.usb-Hercules_Rocksmith_USB_Guitar_Adapter-00`, confirmed via direct `arecord` test). WASAPI/mmdevapi tracing at this stage showed only `render_*` (output) calls, never a single `capture_*` call - PortAudio was not even attempting to open a capture client.

`winecfg`'s Audio tab "Voice input device" dropdown, with the `winealsa` driver active, showed only generic, unnamed entries ("(System default)"/"In: default"); switching to the `winepulse` driver made the dropdown show the device by its real name ("Rocksmith Guitar Adapter Mono"), but this made no difference to PortAudio's in-game enumeration - it still found nothing.

2. Switching wine's ALSA driver: enumeration works, but skips mono devices

Setting `Software\Wine\Drivers\Audio` to `alsa` (via `protontricks 221680 sound=alsa`) changed PortAudio's behavior: it began enumerating 2 devices - "USB Audio - USB Audio #1" (an onboard motherboard USB audio chip, `0b05:1996`, **stereo**, unrelated to the guitar cable) and "default". The actual Real Tone Cable (ALSA card 0, confirmed via `arecord -l`) was **never listed**.

Comparing `/proc/asound/cardX/streamY` for both devices:

Device	Capture channels
Rocksmith USB Guitar Adapter (card 0)	**1 (mono)**
ASUS onboard USB Audio (card 3, the one PortAudio *did* find)	2 (stereo)

This strongly suggests **Wine's ALSA-backed WASAPI capture enumeration silently filters out mono-only USB Audio Class capture devices**, only surfacing stereo-capable ones. Since Real Tone Cables are mono by design (single guitar input channel), this alone would explain total invisibility regardless of driver/registry configuration.

3. Fake-stereo workaround via kernel ALSA loopback: device becomes visible, but still no reaction

To test the mono-filtering hypothesis directly, a kernel-level ALSA loopback device was created:

```
sudo modprobe snd-aloop
```

This creates a genuine ALSA hardware card (`card 4: Loopback`, visible in `/proc/asound/cards` exactly like real hardware). A background bridge process duplicates the cable's mono signal into the loopback's playback side as 2 channels:

```
ffmpeg -f pulse -i alsa_input.usb-
Hercules_Rocksmith_USB_Guitar_Adapter-00.mono-fallback
\
       -ac 2 -f alsa hw:Loopback,0,0
```

The paired capture side (`hw:Loopback,1,0`) was verified, independently of the game, to carry correct duplicated-stereo signal (`arecord` + `ffmpeg -af astats` showing identical, non-silent levels on both channels).

This device was detected and opened by the game (`audiodump.txt`: `Id 11`/`Id 12`, `"In: Loopback - Loopback PCM"`, opened into `Slot 2`/`Slot 3`, `channels 2`, `exclusive[ YES ]`) - unlike an earlier attempt using a purely user-space ALSA PCM definition (`~/.asoundrc`, `type route`/`type pipewire`), which the game's process (running inside the sandboxed **Steam Linux Runtime "Sniper" container**, appid `1628350`) never saw at all, presumably because the container's bundled ALSA libraries don't chain to the host user's `~/.asoundrc` and/or lack the `pipewire` PCM plugin type. A

genuine kernel-level ALSA hardware card, by contrast, is visible inside the container the same way real USB hardware is.

Despite the Loopback device being opened as a valid input slot with confirmed 2-channel signal reaching it (verified live via `/proc/asound/card4/pcm1c/sub0/status` showing `state: RUNNING` with `owner_pid` = the game's own wine process while it was open in exclusive mode) - **the in-game meter showed no reaction whatsoever**, including to a loud, unambiguous 1kHz test tone fed directly into the loopback.

4. Ruling out routing entirely: three independent working paths, zero reactions

At this point the hypothesis was inverted: does the meter react to *anything*? Three independent, separately-verified-working audio paths were tested against the live gain-check/tuner screen:

1. **Loud synthetic 1kHz test tone** fed into the Loopback device (`speaker-test -D hw:Loopback,0,0`), with the game's capture stream confirmed `RUNNING`/exclusively open on the paired capture side. **No reaction.**
2. **Live signal from the real cable**, via the system's actual PipeWire default input (which happened to already be set to the Hercules adapter) - confirmed via `wpctl status` showing an **active** PipeWire connection `ALSA plug-in [wine-preloader] input_MONO < Rocksmith USB Guitar Adapter:capture_MONO`. **No reaction.**
3. **Loud ambient noise** (clapping) directed at the PC's onboard ASUS USB Audio device (the one device the engine treats as `Slot 0`, i.e. its primary/first-found input). **No reaction.**

All three paths were independently confirmed, via host-side tools, to be delivering real, non-silent audio to the exact ALSA/PipeWire node the game's process had open. None produced any visible change in any in-game meter (calibration screen or Tuner).

5. WINEDEBUG trace: Wine's WASAPI implementation is functioning correctly

Steam launch options were temporarily set to:

```
WINEDEBUG=+wasapi,+mmdevapi,+alsa PROTON_LOG=1
PROTON_LOG_DIR=<path> %command%
```

(Steam must be fully shut down, `steam -shutdown`, before editing `localconfig.vdf`'s `LaunchOptions` for the change to persist.)

A ~35-90MB trace was captured across a play session. Findings:

- **183,549** `capture_*` mmdevapi calls logged - `capture_GetBuffer`/ `capture_ReleaseBuffer` cycle continuously and normally throughout the session (no stalls, no repeated failures).
- `alsa:alsa_create_stream` + `alsa:map_channels need_remapping: 0, alsa_channels: 1` confirm Wine opens the **real mono ALSA device** directly, and `alsa:alsa_read_data read 144 from 656 limit 1392` (and many more like it) confirm **genuine, continuous reads** of real audio data - not mocked or empty reads.
- **Zero** occurrences of `AUDCLNT_BUFFERFLAGS_SILENT`.
- **Zero** `err:mmdevapi` lines. The only `fixme:mmdevapi` line in the entire trace is an unrelated, harmless `AEV_SetMasterVolumeLevelScalar stub`.
- Property-store queries (`MMDevPropStore_GetValue`) for device friendly name, audio engine device format, and endpoint form factor all resolve without error.

In short: **every observable Wine-side API interaction is textbook correct.** There is no error, no silence, no failed negotiation anywhere in the trace.

Conclusion

The audio pipeline is provably intact end-to-end: physical cable -> ALSA -> PipeWire -> Wine's ALSA backend -> Wine's WASAPI mmdevapi implementation -> real, continuous, correctly-flagged capture buffers delivered to the game process. This has been independently verified via host tooling (`arecord`, `wpctl`, `/proc/asound` status files) and via Wine's own debug trace. Three different capture devices (2 real cables + 1 synthetic loopback) all fail identically.

The remaining, unexplained failure is that Rocksmith2014.exe's own internal signal-level analysis (whatever it does with the buffers it receives via `IAudioCaptureClient::GetBuffer`) never registers a valid signal, in any of the game's audio-level UI screens. Since Wine's API surface behaves correctly by every external measure available, the leading hypothesis is a **timing/padding/position semantics difference** between Wine's WASAPI capture implementation and real Windows (e.g. `IAudioClient::GetCurrentPadding`, buffer position/ QPC timestamp reporting, or exclusive-mode stream state transitions) that this particular (older, Wwise/PortAudio-based) game's audio engine is sensitive to in a way that doesn't surface as an explicit Wine error.

Suggested next steps for whoever picks this up

- Compare `IAudioClient::GetCurrentPadding`/buffer-position reporting between Wine's mmdevapi/ALSA capture implementation and real Windows WASAPI exclusive-mode capture streams - this is the most likely remaining suspect given everything else traces correctly.
- Consider filing this as a Wine bug (bugs.winehq.org) against `dlls/mmdevapi/dlls/winealsa.drv`, with this document and the raw trace attached. The trace is large (tens of MB); a trimmed excerpt covering just one `OpenStream`/`GetBuffer` cycle would likely suffice for a bug report.
- If pursuing further locally: attach a Windows-side WASAPI trace tool (or find another WASAPI-exclusive-capture Windows game/app with known working Wine behavior) to compare buffer-timing semantics directly.
- Reverse-engineering `Rocksmith2014.exe`'s own audio-analysis routine was considered but not attempted - the executable is closed-source, and this would be a substantial separate undertaking (disassembly, no symbols, Wwise engine internals) with uncertain payoff compared to a Wine-side fix, which would also benefit any other WASAPI-capture- sensitive title.

Appendix: key commands used during investigation

```
# Pin Proton version (fixes unrelated launch crash)
# ~/.local/share/Steam/config/config.vdf ->
CompatToolMapping -> "221680" -> "name" "proton_10"

# Switch wine audio driver to ALSA
protontricks 221680 sound=alsa

# Kernel ALSA loopback setup
sudo modprobe snd-aloop
ffmpeg -f pulse -i alsa_input.usb-
Hercules_Rocksmith_USB_Guitar_Adapter-00.mono-fallback
\
       -ac 2 -f alsa hw:Loopback,0,0

# Verify loopback pairing/signal independently of the
game
arecord -D hw:Loopback,1,0 -f S16_LE -r 48000 -c 2 -d
3 test.wav
```

```
ffmpeg -i test.wav -af "astats=metadata=0" -f null -

# Full WASAPI/ALSA debug trace (Steam launch options,
appid 221680)
WINEDEBUG=+wasapi,+mmdevapi,+alsa PROTON_LOG=1
PROTON_LOG_DIR=/path/to/dir %command%
```